AEGISAI ENTERPRISE EDITION

TECHNICAL DOCUMENTATION SUITE

Development & Maintenance Guide

Coding Standards, Testing Frameworks & Contribution Guidelines

Platform Version:	**2.5.0 Enterprise Edition**
Compliance Scope:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
License:	**Apache License 2.0 (Open Source)**

AEGIS AI ASSURANCE PLATFORM

Code Development and Maintenance Guide

Version: 2.5.0 Enterprise (2026 Edition)

Classification: Technical Guide for Software Developers and AI Engineers

Tech Stack: TypeScript 5.8, React 19, Node.js 22, Express, Vite 6, Tailwind CSS v4, Google GenAI SDK

---

1. Local Development Environment Setup

1.1 Development Requirements

- **Node.js:** Version >= 20.10.0 (Recommended: Node 22.x LTS).
- **Package Manager:** npm >= 10.x.
- **Recommended IDE:** VS Code or Google Antigravity IDE with TypeScript, Tailwind CSS, and ESLint extensions.

1.2 Starting Development Mode (HMR)

```
# 1. Install all dependencies
npm install

# 2. Configure the local environment file
cp .env.example .env
# Set GEMINI_API_KEY with a valid development key

# 3. Start the server in development mode
npm run dev
```

The `npm run dev` command runs `tsx server.ts`, which launches Vite in middleware mode with **Hot Module Replacement (HMR)** at `http://localhost:3000`. Any changes to `.tsx` or `.ts` files will be reflected instantly without restarting the server.

---

2. Codebase Structure and Module Organization

```
AegisAI/
├── config/
│   └── frameworks/            # Local JSON storage for BYOF frameworks (LFPDPPP, LFPC, EU AI Act)
├── docs/                      # Official engineering documentation suite and manuals
├── public/                    # Static assets served at the server root
├── scripts/                   # Automated test suites (OWASP AI, Integration) and PDF generators
├── src/
│   ├── components/
│   │   ├── admin/             # Model and token administration panel (AdminSettingsView)
│   │   ├── api/               # API endpoint documentation and testing (ApiConnectionView)
│   │   ├── architecture/      # Architecture diagrams and specifications (ArchitectureView)
│   │   ├── auth/              # Secure login with Google OAuth (Login)
│   │   ├── dashboard/         # Main views: Dashboard, AuditForm, CatalogView, AuditReportView
│   │   ├── gcp/               # Live Google Cloud scanner (GcpLiveScannerView)
│   │   ├── layout/            # Structural components: Sidebar, BrandAssets
│   │   └── ui/                # Design system: Button, Badge, Card, Input, Label
│   ├── lib/
│   │   ├── actionPlanGenerator.ts # Markdown and PDF action plan generator
│   │   ├── aisvsData.ts       # Canonical definition of the 12 OWASP AI-SVS 1.0 chapters
│   │   ├── data.ts            # Initial data and utilities
│   │   ├── firebase.ts        # Firebase client and Firestore initialization
│   │   ├── iso42001Data.ts    # Canonical definition of ISO/IEC 42001:2023
│   │   ├── remediatedArchitectureGenerator.ts # 100% remediated architecture generator
│   │   ├── sampleAudits.ts    # Pre-configured sample audits
│   │   └── utils.ts           # Helper formatting and styling functions
│   ├── services/
│   │   └── gcpScannerService.ts # Google Cloud APIs live scanning service
│   ├── types.ts               # Core TypeScript types and interface definitions
│   ├── App.tsx                # Main view router with React.lazy and Suspense
│   └── main.tsx               # React application entry point
├── firestore.rules            # Declarative Cloud Firestore security rules
├── server.ts                  # Express backend: Map-Reduce Engine, Gemini Inference, and REST API
├── package.json               # Dependencies and build scripts
├── tsconfig.json              # TypeScript compiler configuration (Strict Mode)
└── vite.config.ts             # Vite bundler and chunk splitting configuration
```

3. Coding Conventions and Best Practices

3.1 Strict Typing in TypeScript

- The use of the `any` type is not permitted within core evaluator functions.
- Every new property added to `AuditReport` or `PuntoProbado` must be formally defined in `src/types.ts`.
- The type-checking command `npm run lint` (`tsc --noEmit`) must be executed and pass with 0 errors before pushing any commit.

3.2 Handling Artificial Intelligence Responses

When invoking Gemini models for structured tasks:

- Always use `responseSchema` with primitive types (`Type.OBJECT`, `Type.ARRAY`, `Type.STRING`).
- Inference parameters must be strictly deterministic:

```
config: {
  temperature: 0.0,
  topK: 1,
  topP: 0.0,
  responseMimeType: 'application/json',
  responseSchema: myStrictSchema
}
```

3.3 Server-Sent Events (SSE) Streaming Management

In endpoints that stream SSE events:

- Always implement the `: keep-alive\n\n` heartbeat using `setInterval` to prevent load balancers or proxies from dropping the connection.
- Capture the `req.on('close')` event to clear timers and prevent memory leaks.

---

4. How to Extend the System

4.1 Incorporating a New Cloud Platform into the Shared Responsibility Matrix

To add a new cloud provider (e.g., *Oracle Cloud Infrastructure - OCI Generative AI* or *IBM watsonx*):

1. Open `src/types.ts` and extend the `TechPlatformKey` type:

```
export type TechPlatformKey = 'gemini-enterprise' | 'vertex-ai' | 'azure-openai' | 'aws-bedroc
```

2. Open `server.ts` and add the entry to the `PLATFORM_PROFILES` dictionary:

```
'oci-ai': {
  name: 'Oracle Cloud OCI Generative AI',
  deploymentType: 'PaaS',
  provider: 'Oracle Cloud Infrastructure',
  inheritedSafeguards: [
    'Isolation of dedicated model inference instances.',
    'SOC 2 Type II certification and ISO 27001 compliance.',
    'Contractual guarantee of no retention of customer prompts for retraining.'
  ],
  exemptChapters: ['C04', 'C06']
}
```

3. Open `src/components/dashboard/AuditForm.tsx` and add the card in Section 3 of the form.

4.2 Extending the Mitigation Recipe Schema (4D)

If you want to add a new field to the action plan (e.g., `impactLevel` or `cveReferences`):

1. Modify the `RemediationDetails` interface in `src/types.ts`.
2. Update the `batchResponseSchema` schema in `server.ts` so that Gemini generates the new field in a structured manner.

3. Update `src/components/dashboard/AuditReportView.tsx` and `src/lib/actionPlanGenerator.ts` to render the new attribute in the UI and in the exportable PDF.

4.3 Adding or Modifying Providers in the Multi-Provider LLM Engine

To support a new protocol or inference router:

1. Modify the `LLMProviderType` type in `src/types.ts`:

```
export type LLMProviderType = 'gemini' | 'openrouter' | 'openai' | 'custom_openai_compatible'
```

2. In `server.ts`, update the dispatcher function `executeAIWithFallback` to connect the corresponding client.
3. In `src/components/admin/AdminSettingsView.tsx`, add the visual tab for the new provider, along with its suggested models and default base URL.
4. Validate the integration by running the test suite:

```
npm run test:llm
```

---

5. Testing and Automated Validation

AegisAI features 4 automated validation suites located in the `scripts/` folder:

```
# 1. Strict TypeScript type checking
npm run lint

# 2. Multi-provider LLM engine and connectivity battery
npm run test:llm

# 3. Execution of the functional and regulatory integration suite
npm run test:integration

# 4. Google Cloud Live Scanner integration tests (GCP Live Scanner)
npm run test:gcp

# 5. Execution of the OWASP AI Top 10 cybersecurity suite
npm run test:security

# 6. Clean production build verification (Frontend + Backend)
npm run build
```

---

6. System Maintenance and Updates

6.1 Google Gemini SDK Update (`@google/genai`)

AegisAI uses the official unified `@google/genai` SDK. To update to newer versions:

```
npm install @google/genai@latest
npm run lint
npm run build
```

After the update, run `/api/admin/test-connection` to validate the compatibility of inference calls.

6.2 Branching Strategy and Version Control (Git Workflow)

- `main`: Protected production branch. It only receives changes validated via Pull Request with all 3 test suites passed (100%).
- `develop`: Integration branch for continuous development.
- `feature/*`: New feature branches (e.g., `feature/iso-5338-integration`).
- `hotfix/*`: Immediate security patches.